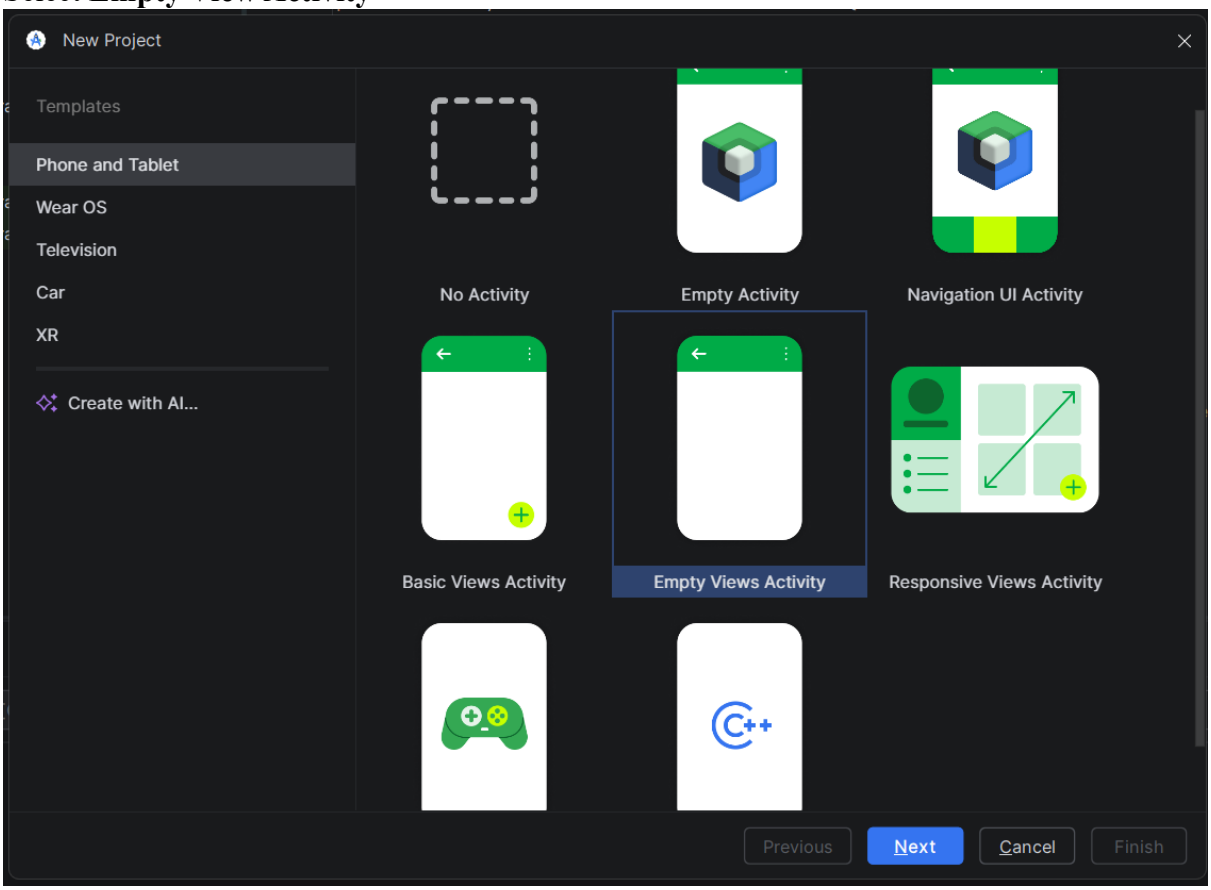
Android Programming

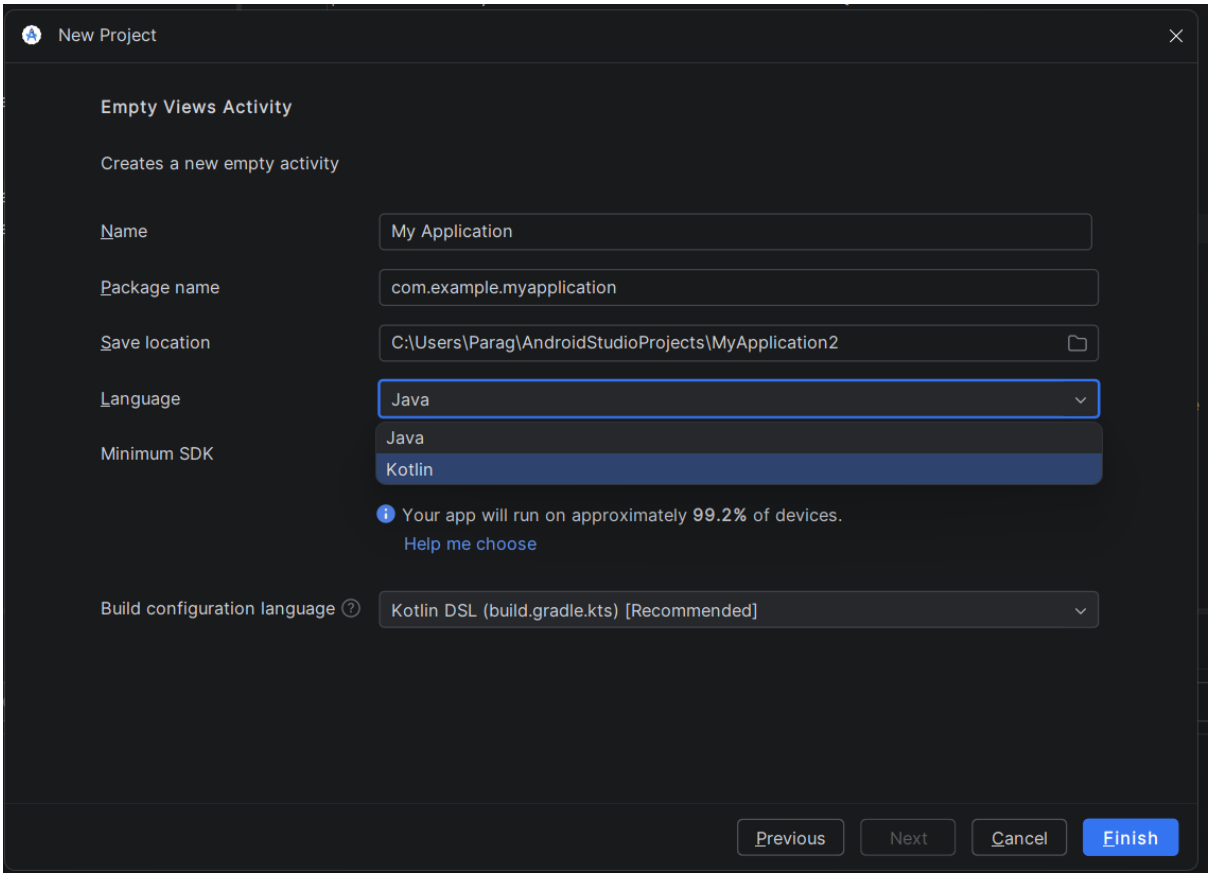
Practical No. 3

Understanding Android Components. Create and Android application with multiple activities. Implement the life cycle methods of each activity and observe their execution.

1. **Create a New Project**
   a. Open **Android Studio**
   b. Click **New Project**
   c. Select **Empty View Activity**



   d. Select **Language → Java**



e. Finish

---

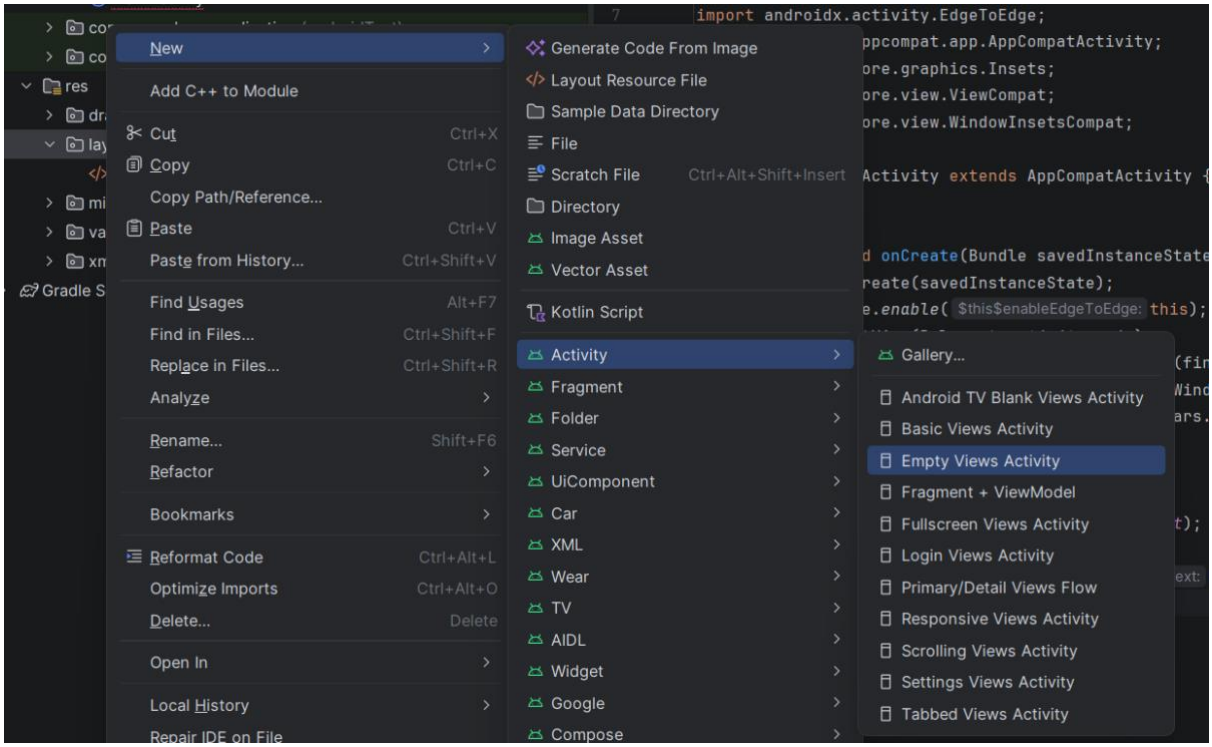
2. **activity_main.xml**

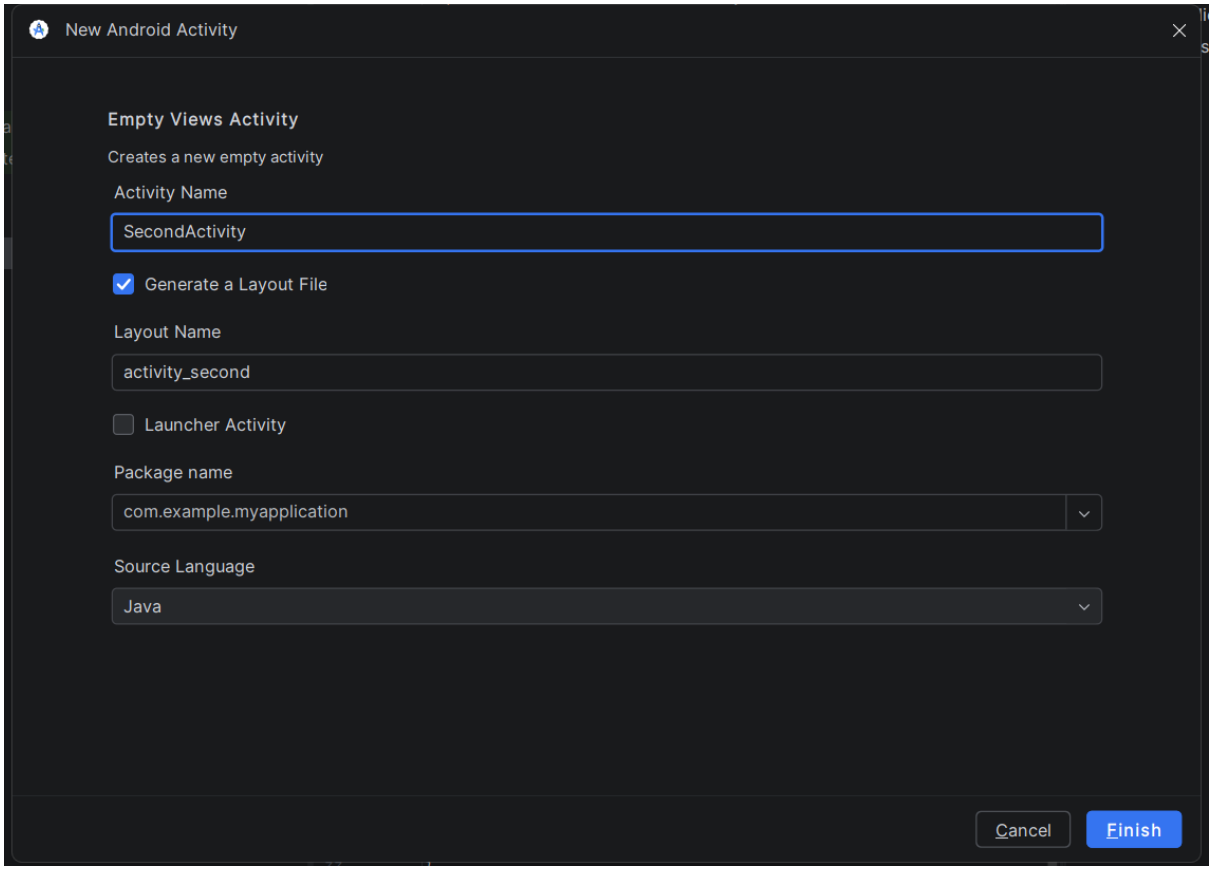
```
<Button
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:id="@+id/btnNext"
    android:text="Next"/>
```

---

3. **Add Second Activity Layout**

Right click on layout→New→Activity→Empty Views Activity





---

4. **Activity_second.xml**

```
<EditText
    android:id="@+id/etUsername"
```

```
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="text"
    android:hint="User Name"
    tools:layout_editor_absoluteX="7dp"
    tools:layout_editor_absoluteY="22dp" />

<EditText
    android:id="@+id/etPassword"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:hint="Password"
    android:inputType="textPassword"
    tools:layout_editor_absoluteX="9dp"
    tools:layout_editor_absoluteY="85dp"
    app:layout_constraintTop_toBottomOf="@+id/etUsername"/>

<Button
    android:id="@+id/btnLogin"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Login"
    tools:layout_editor_absoluteX="56dp"
    tools:layout_editor_absoluteY="139dp"
    app:layout_constraintTop_toBottomOf="@+id/etPassword"/>
```

---

5. **Add Welcome Activity Layout**
   Right click on layout→New→Activity→Empty Views Activity
   activity_welcome.xml

```
<TextView
    android:id="@+id/tvWelcome"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="TextView"
    tools:layout_editor_absoluteX="123dp"
    tools:layout_editor_absoluteY="83dp" />
```

---

6. **MainActivity.java**

```
package com.example.myapplication;

import android.content.Intent;
```

```
import android.os.Bundle;
import android.widget.Button;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
insets) -> {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom);
            return insets;
        });

        Button btnNext = findViewById(R.id.btnNext);
        btnNext.setOnClickListener(v -> {
            Intent intent = new Intent(MainActivity.this, SecondActivity.class);
            startActivity(intent);
        });
        }
    }
```

---

7. **SecondActivity.java**

```
package com.example.myapplication;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
```

```
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class SecondActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_second);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
insets) -> {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom);
            return insets;
        });
        EditText etUsername = findViewById(R.id.etUsername);
        EditText etPassword = findViewById(R.id.etPassword);
        Button btnLogin = findViewById(R.id.btnLogin);
        btnLogin.setOnClickListener(v -> {
            String username = etUsername.getText().toString();
            String password = etPassword.getText().toString();
            // Perform login logic here
            // For example, check if the username and password are correct
            if (username.isEmpty() || password.isEmpty()) {
                // Show an error message if the fields are empty
                Toast.makeText(SecondActivity.this, "Please enter both username and
password", Toast.LENGTH_SHORT).show();
            } else {
                // Login successful, navigate to the next activity
                Intent intent = new Intent(SecondActivity.this, WelcomeActivity.class);
                intent.putExtra("username", username);
                startActivity(intent);
            }
        });
    }
}
```

---

8. **WelcomeActivity.java**

```
package com.example.myapplication;

import android.os.Bundle;
import android.widget.TextView;
```

```
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class WelcomeActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_welcome);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
insets) -> {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom);
            return insets;
        });
        TextView tvWelcome = findViewById(R.id.tvWelcome);
        //Receive username from the intent
        String username = getIntent().getStringExtra("username");
        //Display welcome message with the username
        tvWelcome.setText("Welcome, " + username + "!");
    }
}
```

Practical No. 4

Implementing Services in Android. Develop a service that runs in the background and performs a simple task. Observer the life cycle of the service and its interaction with the main application.

Android Timer Service Example

Objective

Develop a **Service that runs in the background** and performs a **timer task**. The service will count seconds and show them using Toast.

Step 1: Create Android Project

1. Open **Android Studio**
2. Click **New Project**
3. Select **Empty Views Activity**
4. Language: **Java**
5. Project Name: **TimerServiceApp**

Step 2: Create Service Class

Right click **Java Package → New → Service → TimerService**

TimerService.java

```
package com.example.timerserviceapp;

import android.app.Service;
import android.content.Intent;
import android.os.IBinder;
import android.widget.Toast;

public class TimerService extends Service {

    int seconds = 0;
    boolean running = true;

    @Override
    public void onCreate() {
        super.onCreate();
        Toast.makeText(this, "Service Created", Toast.LENGTH_SHORT).show();
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
```

```
        new Thread(new Runnable() {
            @Override
            public void run() {

                while(running)
                {
                    try {
                        Thread.sleep(1000);
                        seconds++;

                        Toast.makeText(getApplicationContext(),
                                "Timer: " + seconds + " seconds",
                                Toast.LENGTH_SHORT).show();

                    } catch (Exception e) {
                        e.printStackTrace();
                    }
                }

            }
        }).start();

        return START_STICKY;
    }

    @Override
    public void onDestroy() {
        running = false;
        Toast.makeText(this,"Service Stopped",Toast.LENGTH_SHORT).show();
        super.onDestroy();
    }

    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
}
```

Step 3: Register Service in AndroidManifest.xml

Add this inside <application>:

```
<service android:name=".TimerService"/>
```

Step 4: Create Layout

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <Button
        android:id="@+id/startService"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Start Timer Service"/>

    <Button
        android:id="@+id/stopService"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Stop Timer Service"/>

</LinearLayout>
```

Step 5: MainActivity.java

```
package com.example.timerserviceapp;

import androidx.appcompat.app.AppCompatActivity;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;

public class MainActivity extends AppCompatActivity {

    Button startService, stopService;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        startService = findViewById(R.id.startService);
```

```
        stopService = findViewById(R.id.stopService);

        startService.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {

                startService(new Intent(MainActivity.this, TimerService.class));
            }
        });

        stopService.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {

                stopService(new Intent(MainActivity.this, TimerService.class));
            }
        });

    }
}
```

Practical No.5

Create a content provider to manage and share data between different components of your android application.

To manage and share data between different components of your Android application, you can implement a **ContentProvider**. A ContentProvider acts as an abstraction layer over your app's data sources (like SQLite databases, files, or even network data) and provides a standard interface for other components or apps to query and modify that data.

Usually it uses **SQLite**, but it is **not mandatory**. You can store data in:

- Arrays
- Lists
- Files
- SharedPreferences
- In-memory data structures

Below is a **simple student-level example of Content Provider without SQLite**, using **ArrayList (in-memory storage)**.

Simple Android Application Using Content Provider (Without SQLite)

1. Create a New Project

1. Open **Android Studio**
2. Click **New Project**
3. Select **Empty View Activity**
4. Select **Language → Java**
5. Finish

---

2. Create Content Provider Class

Right click **package → New → Java Class → MyProvider**

```
package com.example.contentproviderdemo;

import android.content.ContentProvider;

import android.content.ContentValues;

import android.database.Cursor;

import android.database.MatrixCursor;

import android.net.Uri;

import java.util.ArrayList;
```

```
public class MyProvider extends ContentProvider {

    ArrayList<String> list = new ArrayList<>();

    @Override
    public boolean onCreate() {

        list.add("Ram");
        list.add("Shyam");
        list.add("Amit");

        return true;
    }

    @Override
    public Cursor query(Uri uri, String[] projection, String selection,
                        String[] selectionArgs, String sortOrder) {

        MatrixCursor cursor = new MatrixCursor(new String[]{"name"});

        for(String item : list){
            cursor.addRow(new Object[]{item});
        }

        return cursor;
    }

    @Override
    public Uri insert(Uri uri, ContentValues values) {
```

```
        String name = values.getAsString("name");
        list.add(name);
        return uri;
    }

    @Override
    public int delete(Uri uri, String selection, String[] selectionArgs) {
        list.clear();
        return 1;
    }

    @Override
    public int update(Uri uri, ContentValues values, String selection,
                      String[] selectionArgs) {
        return 0;
    }

    @Override
    public String getType(Uri uri) {
        return null;
    }
}
```

---

3. Register Provider in AndroidManifest.xml

```
<provider
    android:name=".MyProvider"
    android:authorities="com.example.provider"
    android:exported="true"/>
```

---

4. activity_main.xml Layout

```
<EditText
android:id="@+id/editText"
android:hint="Enter Name"
android:layout_width="match_parent"
android:layout_height="wrap_content"/>

<Button
android:id="@+id/btnInsert"
android:text="Insert"
android:layout_width="match_parent"
android:layout_height="wrap_content"/>

<Button
android:id="@+id/btnShow"
android:text="Show Data"
android:layout_width="match_parent"
android:layout_height="wrap_content"/>
```

---

5. MainActivity.java

```
package com.example.contentproviderdemo;


import androidx.appcompat.app.AppCompatActivity;


import android.content.ContentValues;

import android.database.Cursor;

import android.net.Uri;

import android.os.Bundle;

import android.view.View;

import android.widget.*;


public class MainActivity extends AppCompatActivity {


    EditText editText;

    Button btnInsert, btnShow;
```

```
Uri uri = Uri.parse("content://com.example.provider");

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    editText = findViewById(R.id.editText);
    btnInsert = findViewById(R.id.btnInsert);
    btnShow = findViewById(R.id.btnShow);

    btnInsert.setOnClickListener(v -> {

        ContentValues values = new ContentValues();
        values.put("name", editText.getText().toString());

        getContentResolver().insert(uri, values);

        Toast.makeText(this,"Data Inserted",Toast.LENGTH_SHORT).show();
    });

    btnShow.setOnClickListener(v -> {

        Cursor cursor = getContentResolver().query(uri,null,null,null,null);

        String result="";

        while(cursor.moveToNext()){
            result += cursor.getString(0) + "\n";
```

```
            }

            Toast.makeText(this,result,Toast.LENGTH_LONG).show();
        });
    }
}
```

Practical No. 6

Implementing broadcast receivers. Design a broadcast receiver to respond to system or application-level broadcast. Test the receiver with different broadcast scenarios.

A **Broadcast Receiver** is an Android component that listens for **broadcast messages** sent by the Android system or other applications.

Example broadcasts:

- Battery low
- Network change
- Device boot completed
- Custom application broadcast

When the broadcast occurs, the receiver **triggers the onReceive() method**.

Types of Broadcast

1. System Broadcast
2. Custom Broadcast

Step 1: Create Android Project

1. Open **Android Studio**
2. Click **New Project**
3. Select **Empty Activity**
4. Language: **Java**
5. Finish

Step 2: Create BroadcastReceiver Class

Right Click
app → java → package name → New → Java Class
Name it: MyReciever

```
package com.example.broadcastdemo;



import android.content.BroadcastReceiver;

import android.content.Context;

import android.content.Intent;

import android.widget.Toast;
```

```
public class MyReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context,
                "Power Connected Broadcast Received",
                Toast.LENGTH_LONG).show();
    }
}
```

Step 3: Register Receiver in AndroidManifest.xml

Open **AndroidManifest.xml**

Add receiver inside <application> tag.

```
<receiver android:name=".MyReceiver">
    <intent-filter>
        <action android:name="android.intent.action.ACTION_POWER_CONNECTED"/>
    </intent-filter>
</receiver>
```

Step 4: Run the Application

1. Connect Android phone/emulator
2. Run the app

Step 5: Test Broadcast Scenario

Scenario 1: Power Connected

Plug the charger.

Result:

Scenario 2: Power Disconnected

Modify Manifest:

<action android:name="android.intent.action.ACTION_POWER_DISCONNECTED"/>

Result:
When charger removed → Toast appears.

Practical No. 7

Designing use interfaces. Create an android application with various layouts, including linear, relative, and constraint layouts. Implement input controls such as text input, checkboxes, and radio buttons in your UI.

Types of Layouts in Android

1. Linear Layout

LinearLayout arranges elements in a single direction either vertical or horizontal.

Features

- Simple layout
- Arranges components in one line
- Uses orientation property

2. Relative Layout

RelativeLayout places UI components relative to other components or parent layout.

Features

- Position controls relative to each other
- Flexible layout design

3. Constraint Layout

ConstraintLayout is the most modern and flexible layout used in Android.

Features

- Allows complex UI design
- Reduces nested layouts
- Uses constraints (top, bottom, start, end)

1. Objective

Create an Android application that demonstrates different layouts (**LinearLayout, RelativeLayout, ConstraintLayout**) and uses input controls like **EditText, CheckBox, and RadioButton**.

---

2. Create New Project

1. Open **Android Studio**
2. Click **New Project**
3. Select **Empty Views Activity**
4. Language: **Java**
5. Click **Finish**

---

3. Layout Design (activity_main.xml)

Open **res → layout → activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- Linear Layout -->
    <LinearLayout
        android:id="@+id/linearLayout"
        android:orientation="vertical"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:padding="16dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">

        <TextView
            android:text="Enter Name"
            android:textSize="18sp"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>

        <EditText
            android:id="@+id/editName"
```

```
        android:hint="Your Name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    </LinearLayout>

    <!-- Relative Layout -->
    <RelativeLayout
      android:id="@+id/relativeLayout"
      android:layout_width="0dp"
      android:layout_height="wrap_content"
      android:padding="16dp"
      app:layout_constraintTop_toBottomOf="@id/linearLayout"
      app:layout_constraintStart_toStartOf="parent"
      app:layout_constraintEnd_toEndOf="parent">

      <CheckBox
        android:id="@+id/checkJava"
        android:text="Java"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

      <CheckBox
        android:id="@+id/checkAndroid"
        android:text="Android"
        android:layout_below="@id/checkJava"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    </RelativeLayout>

    <!-- Radio Buttons -->
    <RadioGroup
      android:id="@+id/radioGroup"
      android:layout_width="0dp"
      android:layout_height="wrap_content"
      android:orientation="vertical"
      android:padding="16dp"
      app:layout_constraintTop_toBottomOf="@id/relativeLayout"
      app:layout_constraintStart_toStartOf="parent"
      app:layout_constraintEnd_toEndOf="parent">

      <RadioButton
        android:id="@+id/radioMale"
        android:text="Male"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

      <RadioButton
        android:id="@+id/radioFemale"
```

```
        android:text="Female"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    </RadioGroup>

    <Button
        android:id="@+id/btnSubmit"
        android:text="Submit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@id/radioGroup"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Result"
    android:id="@+id/tvResults"
    app:layout_constraintTop_toBottomOf="@+id/btnSubmit"/>



</androidx.constraintlayout.widget.ConstraintLayout>
```

---

4. Java Code (MainActivity.java)

Open **MainActivity.java**

```
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.RadioButton;
import android.widget.RadioGroup;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {
```

```
    EditText editName;
    CheckBox checkJava, checkAndroid;
    RadioGroup radioGroup;
    Button btnSubmit;
    TextView tvResult;


    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) ->
{
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        editName = findViewById(R.id.editName);
        checkJava = findViewById(R.id.checkJava);
        checkAndroid = findViewById(R.id.checkAndroid);
        radioGroup = findViewById(R.id.radioGroup);
        btnSubmit = findViewById(R.id.btnSubmit);
        tvResult= findViewById(R.id.tvResults);


        btnSubmit.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {

                String name = editName.getText().toString();

                String skills = "";
                if(checkJava.isChecked())
                    skills += "Java ";

                if(checkAndroid.isChecked())
                    skills += "Android";

                int selectedId = radioGroup.getCheckedRadioButtonId();
                RadioButton radioButton = findViewById(selectedId);

                String gender = radioButton.getText().toString();

                String result = "Name: " + name +
                        "\nSkills: " + skills +
                        "\nGender: " + gender;
                tvResult.setText(result.toString());
                //Toast.makeText(MainActivity.this, result, Toast.LENGTH_LONG).show();
```

```
        }
    });
  }
}
```

---

5. Output

The application screen will include:

- **Text Input** → Enter Name
- **CheckBoxes** → Select Skills (Java / Android)
- **Radio Buttons** → Select Gender
- **Submit Button** → Displays result using TextView

Practical No. 8

Building dialogs and Pickers. Develop an android application that utilizes dialog boxes, date pickers, and time pickers. Implement functionality to handle user interactions with these UI components.

Aim

To develop an Android application that uses **Alert Dialog, Date Picker, and Time Picker** and handles user interaction.

1. AlertDialog Class

AlertDialog is a dialog box that displays a message to the user and asks for a response.
It usually contains **buttons like Yes, No, OK, Cancel**.

It is used when the application needs **confirmation or decision from the user**.

Example

When the user clicks **Show Dialog**, this dialog appears:

Do you want to continue?
[YES] [NO]

Syntax Example

```
AlertDialog.Builder builder = new AlertDialog.Builder(MainActivity.this);

builder.setTitle("Confirmation");
builder.setMessage("Do you want to continue?");

builder.setPositiveButton("Yes", (dialog, which) -> {
   Toast.makeText(MainActivity.this,"You clicked YES",Toast.LENGTH_SHORT).show();
});
```

```
builder.setNegativeButton("No", (dialog, which) -> {
   Toast.makeText(MainActivity.this,"You clicked NO",Toast.LENGTH_SHORT).show();
});

builder.show();
```

Important Methods

Method	Description
setTitle()	Sets the title of the dialog
setMessage()	Displays the message
setPositiveButton()	Action when user clicks positive button
setNegativeButton()	Action when user clicks negative button
show()	Displays the dialog

2. DatePickerDialog Class

DatePickerDialog is used to allow the user to **select a date from a calendar interface**.

It displays a **calendar style UI** where users choose **day, month, and year**.

Example

When the user clicks **Select Date**, a calendar opens and the user selects a date.

Example output:

Selected Date: 15/03/2026

Syntax Example

```
Calendar calendar = Calendar.getInstance();

int year = calendar.get(Calendar.YEAR);
int month = calendar.get(Calendar.MONTH);
int day = calendar.get(Calendar.DAY_OF_MONTH);

DatePickerDialog datePickerDialog = new DatePickerDialog(
      MainActivity.this,
      (view, year1, month1, dayOfMonth) -> {
         txtResult.setText("Selected Date: " + dayOfMonth + "/" + (month1+1) + "/" + year1);
      },
      year, month, day
);

datePickerDialog.show();
```

Important Parameters

Parameter	Description
context	Current activity
listener	Handles date selection
year	Default year
month	Default month
day	Default day

3. TimePickerDialog Class

TimePickerDialog allows the user to **select time (hour and minute)**.

It displays a **clock interface** for selecting time.

Example

When user clicks **Select Time**, the clock interface opens.

Example output:

Selected Time: 14:30

Syntax Example

```
Calendar calendar = Calendar.getInstance();

int hour = calendar.get(Calendar.HOUR_OF_DAY);
int minute = calendar.get(Calendar.MINUTE);

TimePickerDialog timePickerDialog = new TimePickerDialog(
        MainActivity.this,
        (view, hourOfDay, minute1) -> {
            txtResult.setText("Selected Time: " + hourOfDay + ":" + minute1);
        },
        hour,
        minute,
        true
);

timePickerDialog.show();
```

Important Parameters

Parameter	Description
context	Current activity

Parameter	Description
listener	Handles time selection
hour	Default hour
minute	Default minute
is24HourView	True for 24-hour format

---

1. XML Layout (activity_main.xml)

Create UI with three buttons and one TextView.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <Button
        android:id="@+id/btnDialog"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Show Dialog"/>

    <Button
        android:id="@+id/btnDate"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Select Date"
        android:layout_marginTop="20dp"/>

    <Button
        android:id="@+id/btnTime"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Select Time"
        android:layout_marginTop="20dp"/>

    <TextView
        android:id="@+id/txtResult"
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:text="Result will appear here"
        android:textSize="18sp"
        android:layout_marginTop="30dp"/>

</LinearLayout>
```

---

2. MainActivity.java

```
package com.example.dialogpicker;

import androidx.appcompat.app.AppCompatActivity;

import android.app.AlertDialog;
import android.app.DatePickerDialog;
import android.app.TimePickerDialog;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import java.util.Calendar;

public class MainActivity extends AppCompatActivity {

    Button btnDialog, btnDate, btnTime;
    TextView txtResult;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnDialog = findViewById(R.id.btnDialog);
        btnDate = findViewById(R.id.btnDate);
        btnTime = findViewById(R.id.btnTime);
        txtResult = findViewById(R.id.txtResult);

        // Alert Dialog
        btnDialog.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {

                AlertDialog.Builder builder = new AlertDialog.Builder(MainActivity.this);
                builder.setTitle("Confirmation");
```

```
            builder.setMessage("Do you want to continue?");

            builder.setPositiveButton("Yes", (dialog, which) -> {
                Toast.makeText(MainActivity.this,"You clicked
YES",Toast.LENGTH_SHORT).show();
            });

            builder.setNegativeButton("No", (dialog, which) -> {
                Toast.makeText(MainActivity.this,"You clicked
NO",Toast.LENGTH_SHORT).show();
            });

            builder.show();
        }
    });

    // Date Picker
    btnDate.setOnClickListener(view -> {

        Calendar calendar = Calendar.getInstance();

        int year = calendar.get(Calendar.YEAR);
        int month = calendar.get(Calendar.MONTH);
        int day = calendar.get(Calendar.DAY_OF_MONTH);

        DatePickerDialog datePickerDialog = new DatePickerDialog(
                MainActivity.this,
                (view1, year1, month1, dayOfMonth) -> {
                    txtResult.setText("Selected Date: " + dayOfMonth + "/" + (month1+1) + "/" +
year1);
                },
                year, month, day
        );

        datePickerDialog.show();
    });

    // Time Picker
    btnTime.setOnClickListener(view -> {

        Calendar calendar = Calendar.getInstance();

        int hour = calendar.get(Calendar.HOUR_OF_DAY);
        int minute = calendar.get(Calendar.MINUTE);

        TimePickerDialog timePickerDialog = new TimePickerDialog(
```

```
                MainActivity.this,
                (view12, hourOfDay, minute1) -> {
                    txtResult.setText("Selected Time: " + hourOfDay + ":" + minute1);
                },
                hour,
                minute,
                true
            );

            timePickerDialog.show();
        });

    }
}
```

Practical No. 9

Working with Intents. Create an android application that uses both implicit and explicit intents to navigate between different activities. Implement intent filters for specific actions.

Aim

To create an Android application that uses **Explicit and Implicit Intents** to navigate between activities and implement **Intent Filters**.

---

1. What is an Intent?

An **Intent** is a messaging object used to request an action from another component in Android.

Intents are mainly used for:

- Starting another activity
- Opening system applications
- Passing data between activities

---

2. Types of Intents

1️⃣ Explicit Intent

Explicit intent specifies the **exact activity** to start.

Example:

- Move from **MainActivity → SecondActivity**

2️⃣ Implicit Intent

Implicit intent **does not specify the target activity**.
Android finds a suitable application to perform the action.

Example:

- Open browser
- Open dialer
- Share text

---

3. Project Structure

MainActivity
SecondActivity
activity_main.xml
activity_second.xml
AndroidManifest.xml

---

4. activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center">

    <Button
        android:id="@+id/btnExplicit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Open Second Activity"/>

    <Button
        android:id="@+id/btnBrowser"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Open Website"
        android:layout_marginTop="20dp"/>

    <Button
        android:id="@+id/btnDial"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Open Dialer"
```

```
        android:layout_marginTop="20dp"/>

</LinearLayout>
```

---

5. MainActivity.java

```
package com.example.intentexample;

import androidx.appcompat.app.AppCompatActivity;

import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.widget.Button;

public class MainActivity extends AppCompatActivity {

    Button btnExplicit, btnBrowser, btnDial;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnExplicit = findViewById(R.id.btnExplicit);
        btnBrowser = findViewById(R.id.btnBrowser);
        btnDial = findViewById(R.id.btnDial);

        // Explicit Intent
        btnExplicit.setOnClickListener(v -> {
            Intent intent = new Intent(MainActivity.this, SecondActivity.class);
            startActivity(intent);
        });

        // Implicit Intent - Open Website
        btnBrowser.setOnClickListener(v -> {
            Intent intent = new Intent(Intent.ACTION_VIEW);
            intent.setData(Uri.parse("https://www.google.com"));
            startActivity(intent);
        });

        // Implicit Intent - Open Dialer
        btnDial.setOnClickListener(v -> {
            Intent intent = new Intent(Intent.ACTION_DIAL);
            intent.setData(Uri.parse("tel:1234567890"));
            startActivity(intent);
```

```
        });
    }
}
```

---

6. activity_second.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome to Second Activity"
        android:textSize="22sp"/>
</LinearLayout>
```

---

7. SecondActivity.java

```
package com.example.intentexample;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;

public class SecondActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
    }
}
```

---

8. Intent Filter (AndroidManifest.xml)

Intent filters allow an activity to respond to **specific actions**.

Example: open activity when user clicks a link.

```
<activity android:name=".SecondActivity">

    <intent-filter>
        <action android:name="android.intent.action.VIEW"/>

        <category android:name="android.intent.category.DEFAULT"/>
```

```
        <category android:name="android.intent.category.BROWSABLE"/>

        <data android:scheme="https"
            android:host="www.example.com"/>
    </intent-filter>

</activity>
```

This means:

If user opens:

https://www.example.com

Android can launch **SecondActivity**.

---

9. Output Functionality

Explicit Intent

Button click:

MainActivity → SecondActivity

Implicit Intent (Browser)

Button click:

Open Website → Browser opens google.com

Implicit Intent (Dialer)

Button click:

Dialer opens with number 1234567890

Practical No. 10

Implementing Notification. Build an android application that sends notification to the user. Customize the appearance and behaviour of the notifications.

Step 1: Create New Project

1. **Open Android Studio**
2. **Click New Project**
3. **Select Empty Views Activity**
4. **Name: NotificationApp**

5. **Language: Java (or Kotlin if you prefer)**
6. **Click Finish**

Step 2: Open res/layout/activity_main.xml

```
<Button
android:id="@+id/btnNotify"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Send Notification"
android:layout_centerInParent="true"/>
```

Step 3: Add Notification Code in MainActivity

```
import android.app.NotificationChannel;

import android.app.NotificationManager;

import android.os.Build;

import android.os.Bundle;

import android.view.View;

import android.widget.Button;


import androidx.appcompat.app.AppCompatActivity;

import androidx.core.app.NotificationCompat;


public class MainActivity extends AppCompatActivity {


    Button btnNotify;

    String CHANNEL_ID = "my_channel_id";


    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);


        btnNotify = findViewById(R.id.btnNotify);
```

```
        createNotificationChannel();

        btnNotify.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {

                NotificationCompat.Builder builder =
                        new NotificationCompat.Builder(MainActivity.this, CHANNEL_ID)
                                .setSmallIcon(android.R.drawable.ic_dialog_info)
                                .setContentTitle("Hello Student 🎓")
                                .setContentText("This is your first notification!")
                                .setPriority(NotificationCompat.PRIORITY_HIGH);

                NotificationManager manager =
                        (NotificationManager)
getSystemService(NOTIFICATION_SERVICE);

                manager.notify(1, builder.build());
            }
        });
    }

    private void createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {

            NotificationChannel channel = new NotificationChannel(
                    CHANNEL_ID,
                    "My Channel",
                    NotificationManager.IMPORTANCE_HIGH
            );
```

```
            NotificationManager manager =
        getSystemService(NotificationManager.class);

            manager.createNotificationChannel(channel);

        }

    }

}
```

Step 4: Customize Notification

You can customize like this:

```
.setContentTitle("New Message 📩")
.setContentText("You have a new notification")
.setSmallIcon(android.R.drawable.star_on)
.setAutoCancel(true)
```

Add Sound / Vibration:

```
.setDefaults(NotificationCompat.DEFAULT_ALL)
```

Step 5: Add this in AndroidManifest.xml

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

Note: - If nothing is displayed when you click the “Send Notification” button, don’t worry—this is a very common issue

App Notifications Disabled (Device Setting)

👉 **Sometimes phone blocks notifications.**

✓ **Fix:**

1. **Go to Settings**
2. **Apps → Your App**
3. **Notifications → Turn ON**

***Use of POST_NOTIFICATIONS permission**

This permission allows your app to “Send notifications to the user”

***Why is it important?**

Starting from Android 13 (API 33), Apps cannot show notifications without user permission. User must allow notification manually.

***What happens without this permission?**

Notification code runs but nothing is displayed.

Practical No. 11

Introduction to SQLite database. Set up an SQLite database in your android application. Create a table and perform basic CRUD operations.

1. What is SQLite?

SQLite is a lightweight database built into Android.

- No internet needed
- Stores data locally (inside the app)
- Used for apps like contacts, notes, etc.

---

2. Create New Android Project

1. Open **Android Studio**
2. Click **New Project**
3. Choose **Empty Views Activity**
4. Name: SQLiteStudentApp
5. Language: **Java**
6. Click **Finish**

---

3. Create Database Helper Class

Right-click → New → Java Class → DatabaseHelper

```
import android.content.Context;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class DatabaseHelper extends SQLiteOpenHelper {

    public static final String DATABASE_NAME = "Student.db";
    public static final String TABLE_NAME = "student";
    public static final String COL_1 = "ID";
    public static final String COL_2 = "NAME";
    public static final String COL_3 = "MARKS";

    public DatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
```

```
        db.execSQL("CREATE TABLE " + TABLE_NAME +
                "(ID INTEGER PRIMARY KEY AUTOINCREMENT, NAME TEXT, MARKS
INTEGER)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
        onCreate(db);
    }
}
```

Explanation:

- SQLiteOpenHelper → Helps create/manage database
- onCreate() → Runs first time → creates table
- onUpgrade() → Updates database when version changes

---

4. Add CRUD Methods

Add below methods inside DatabaseHelper class

Insert Data (Create)

```
import android.content.ContentValues;

public boolean insertData(String name, String marks) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();

    contentValues.put(COL_2, name);
    contentValues.put(COL_3, marks);

    long result = db.insert(TABLE_NAME, null, contentValues);
    return result != -1;
}
```

Explanation:

- ContentValues → stores data in key-value form
- insert() → inserts row into table

View Data (Read)

```
import android.database.Cursor;

public Cursor getAllData() {
    SQLiteDatabase db = this.getWritableDatabase();
    return db.rawQuery("SELECT * FROM " + TABLE_NAME, null);
}
```

Explanation:

- Cursor → used to read data row by row
- rawQuery() → executes SQL query

Update Data

```
public boolean updateData(String id, String name, String marks) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();

    contentValues.put(COL_1, id);
    contentValues.put(COL_2, name);
    contentValues.put(COL_3, marks);

    db.update(TABLE_NAME, contentValues, "ID = ?", new String[]{id});
    return true;
}
```

Delete Data

```
public Integer deleteData(String id) {
    SQLiteDatabase db = this.getWritableDatabase();
    return db.delete(TABLE_NAME, "ID = ?", new String[]{id});
}
```

---

5. Design UI (activity_main.xml)

```
<LinearLayout
    android:orientation="vertical"
    android:padding="16dp"
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText android:id="@+id/editName"
        android:hint="Enter Name"
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"/>

    <EditText android:id="@+id/editMarks"
        android:hint="Enter Marks"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <EditText android:id="@+id/editId"
        android:hint="Enter ID (for Update/Delete)"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button android:id="@+id/btnAdd"
        android:text="Add Data"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button android:id="@+id/btnView"
        android:text="View Data"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button android:id="@+id/btnUpdate"
        android:text="Update Data"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button android:id="@+id/btnDelete"
        android:text="Delete Data"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

---

6. MainActivity Code

```
import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    DatabaseHelper myDb;
    EditText editName, editMarks, editId;
```

```
Button btnAdd, btnView, btnUpdate, btnDelete;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    myDb = new DatabaseHelper(this);

    editName = findViewById(R.id.editName);
    editMarks = findViewById(R.id.editMarks);
    editId = findViewById(R.id.editId);

    btnAdd = findViewById(R.id.btnAdd);
    btnView = findViewById(R.id.btnView);
    btnUpdate = findViewById(R.id.btnUpdate);
    btnDelete = findViewById(R.id.btnDelete);

    // Add Data
    btnAdd.setOnClickListener(v -> {
        boolean isInserted = myDb.insertData(
                editName.getText().toString(),
                editMarks.getText().toString()
        );

        Toast.makeText(this,
                isInserted ? "Data Inserted" : "Failed",
                Toast.LENGTH_SHORT).show();
    });

    // View Data
    btnView.setOnClickListener(v -> {
        Cursor res = myDb.getAllData();

        if (res.getCount() == 0) {
            showMessage("Error", "No Data Found");
            return;
        }

        StringBuilder buffer = new StringBuilder();
        while (res.moveToNext()) {
            buffer.append("ID: ").append(res.getString(0)).append("\n");
            buffer.append("Name: ").append(res.getString(1)).append("\n");
            buffer.append("Marks: ").append(res.getString(2)).append("\n\n");
        }
```

```
        showMessage("Data", buffer.toString());
    });

    // Update
    btnUpdate.setOnClickListener(v -> {
        boolean isUpdated = myDb.updateData(
                editId.getText().toString(),
                editName.getText().toString(),
                editMarks.getText().toString()
        );

        Toast.makeText(this,
                isUpdated ? "Updated" : "Failed",
                Toast.LENGTH_SHORT).show();
    });

    // Delete
    btnDelete.setOnClickListener(v -> {
        int deletedRows = myDb.deleteData(editId.getText().toString());
        Toast.makeText(this,
                deletedRows > 0 ? "Deleted" : "Not Found",
                Toast.LENGTH_SHORT).show();
    });
  }

  public void showMessage(String title, String message) {
    new android.app.AlertDialog.Builder(this)
            .setTitle(title)
            .setMessage(message)
            .show();
  }
}
```